

Средства симуляции ЦП и ОС

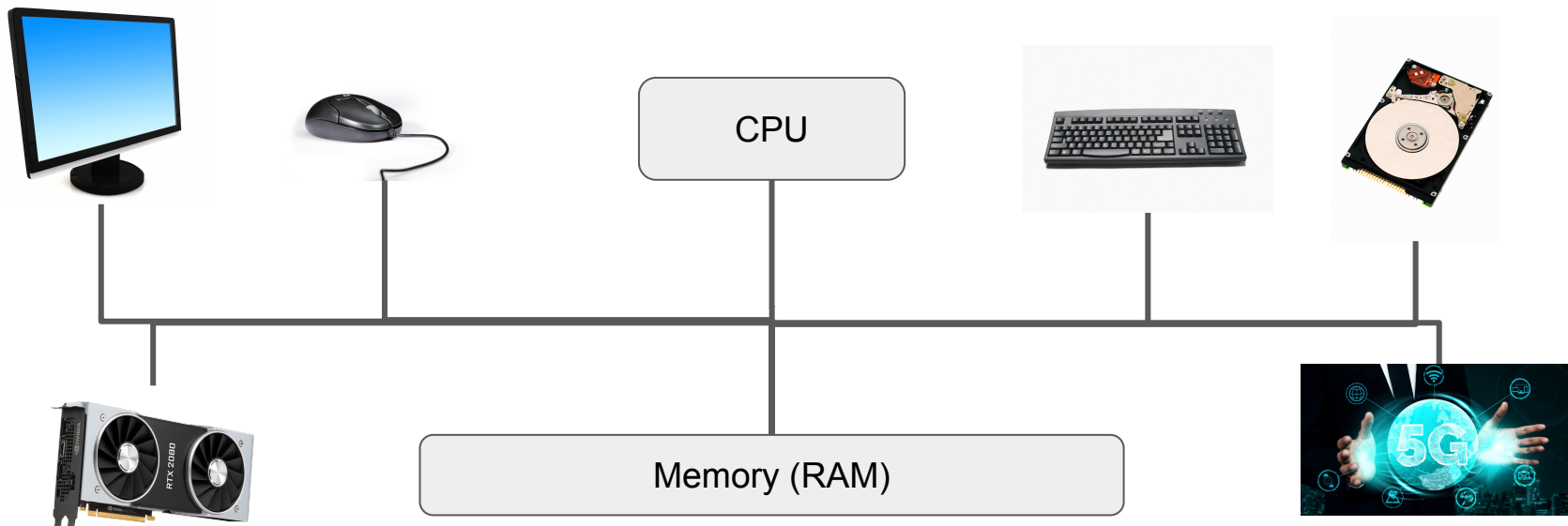
Лекция №18



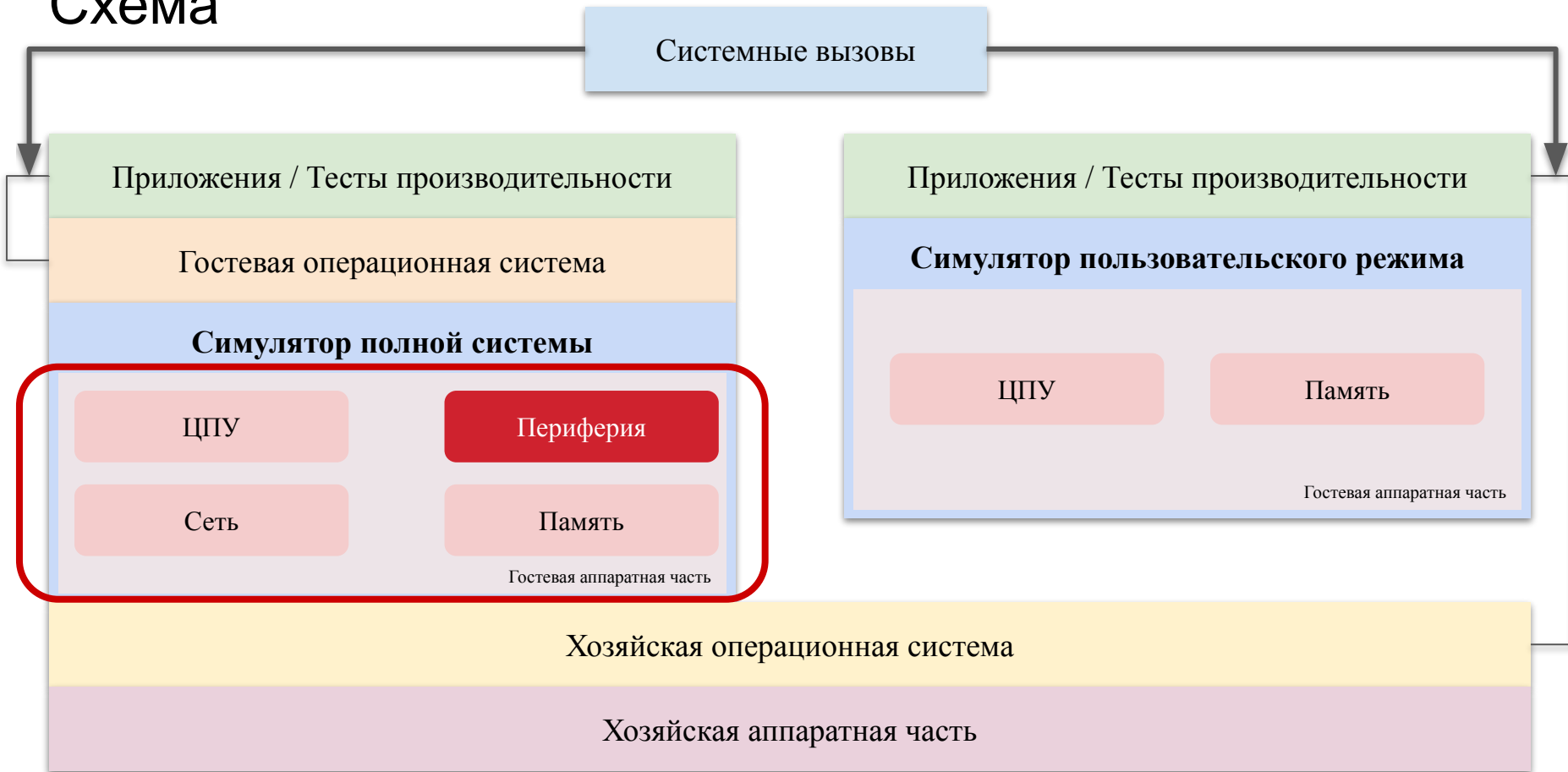
Державин Андрей
Шурыгин Антон

Вспоминаем прошлый семестр

- Современные вычислительные системы это не только CPU
 - И не только один CPU
 - А еще *PU
 - Или даже не PU...
- Симуляция Полной Платформы (Full System Simulation)

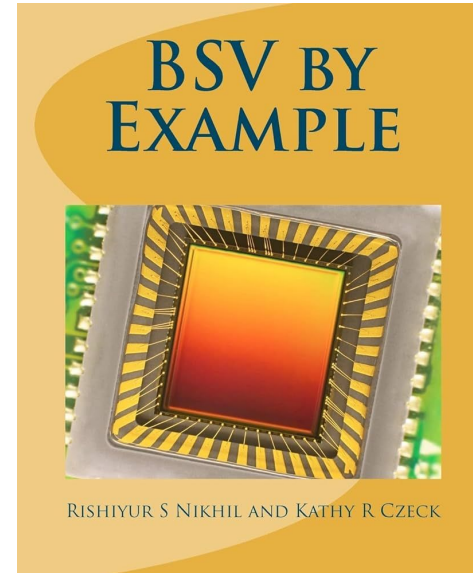


Схема



Языки разработки моделей

When someone comes to you with “a new language” it is often time to run the other way.



Симуляторы

- Симуляторы - большие и сложные системы
- Примерные части:
 - Подсистема плагинов
 - Ядро симуляции
 - Модели устройств
 - И многое другое
- Как описать модели устройств?

Модели устройств

- Два подхода к созданию моделей:
- Библиотеки ЯП общего назначения
 - Используется привычный язык
 - Абстракции организованы как часть библиотеки
 - Пример: SystemC
- Специализированные языки (DSL)
 - Абстракции “защиты” непосредственно в язык
 - Меньший объём кода, выше темп разработки
 - Пример: Simics DML (Device Modeling Language)

А можно по простому?

- Какие преимущества и недостатки у библиотечных решений?
- +:
 - Гибкость - возможность выразить любую логику
 - Готовый тулчейн (компилятор, отладчик)
- -:
 - Может не хватать выразительности
 - Отладка

DSL

- Решает все проблемы?
- +:
 - Концепции области вшиты в язык
 - Более компактный код
- -:
 - Возможности языка ограничены
 - Может потребоваться время на освоение

Integrated DSL

- В какую категорию отнести DSL на основе готового языка?

Требования

- Что мы хотим от ЯП в целом?
- Упрощение разработки через введение удобных концепций
- Язык С
 - Именованная переменная
 - Типы
 - Функции
 - ...

Требования

- Сигналы (фронт)
- Шины
- Операции над битами
- Транзакции
- Расширение уровней сигналов (Z, X)
- Хранение данных (регистры, каналы)
- Карты памяти
- Виртуальное время
- Параллельная обработка

SystemC

- SystemC – библиотека классов C++ для моделирования системного уровня, построенная на основе DES
- Разработка от Accellera и является стандартом, IEEE 1666-2011
- Большое количество компаний-разработчиков аппаратуры выпустили свои реализации
- Позволяет описывать как низкоуровневые детали (RTL) с потактовой точностью, так и высокоуровневые транзакционные модели (TLM)

SystemC

- Базовые аппаратные абстракции:
 - Порты (sc_in, sc_out)
 - Сигналы sc_signal
 - Битовые векторы (sc_bv<>)
 - Четырехзначная логика (sc_lv<>)
- Чувствительность (sensitivity) — это механизм, который определяет, по каким событиям ядро симуляции должно запускать SC_METHOD или SC_THREAD

```

#include "systemc.h"

Декларация модуля (класса) — SC_MODULE(adder)
{
    Порты — sc_in<int> a, b;
             sc_out<int> sum;

    Процесс — void do_add()
              {
                sum = a + b;
              }

    Конструктор — SC_CTOR(adder)
                  {

    Регистрация do_add — SC_METHOD(do_add);
    Список “чувствительности” — sensitive << a << b;
                              }
                              };
  
```

SystemC. Тактовый генератор

```
SC_MODULE(Input) {  
    sc_in<bool> clk;  
    SC_CTOR(Input) {  
        sensitive << clk.pos();  
    }  
};  
  
int sc_main(int argc, char* argv[]) {  
    sc_clock my_clock("my_clock", 10, SC_NS);  
    Input inp0("inp0");  
    inp0.clk(my_clock);  
    sc_trace_file* tf = sc_create_vcd_trace_file("waveform");  
    sc_trace(tf, my_clock, "my_clock_signal");  
    sc_start(40, SC_NS);  
    sc_close_vcd_trace_file(tf);  
    return 0;  
}
```

Вход для тактового сигнала

Вход для тактового сигнала

Создаем экземпляр
провода

Соединяем генератор и
провод

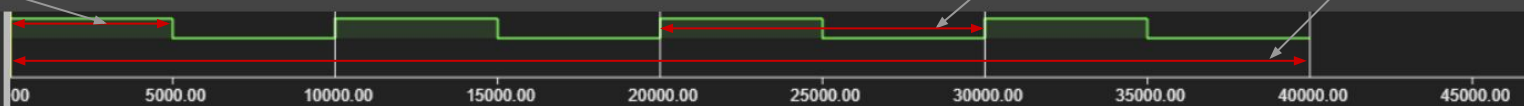
Подаём тактовый генератор
на трассировку

Тактовый
генератор,
T=10 ns

Длительность
симуляции

Длительность
импульса

SystemC.my_clock_signal 1



SystemC. Таймер

- Реализуем простой таймер
 - Каждые четыре такта генерируем импульс на out
 - Выходной сигнал out, поднимается в “1” на один такт раз в период

Тактовый сигнал
Выходной сигнал
Счетчик

Переполнение

Сбрасываем флаг

Чувствительность к
положительному фронту

Подключаем выход

Подключаем вход

Трассируем выход

```
SC_MODULE(Timer) {  
    sc_in<bool> clk;  
    sc_out<bool> out;  
    int count;  
  
    void count_proc() {  
        count++;  
        if (count ≥ 4) {  
            out.write(true);  
            count = 0;  
        } else  
            out.write(false);  
    }  
  
    SC_CTOR(Timer) : count(0) {  
        SC_METHOD(count_proc);  
        sensitive << clk.pos();  
    }  
};  
  
int sc_main(int argc, char* argv[]) {  
    // ...  
    Timer timer("timer");  
    sc_signal<bool> out_sig("out_sig");  
    timer.out(out_sig);  
    timer.clk(clock);  
    // ...  
    sc_trace(tf, clock, "clock");  
    sc_trace(tf, out_sig, "counter_out");  
    // ...  
    return 0;  
}
```

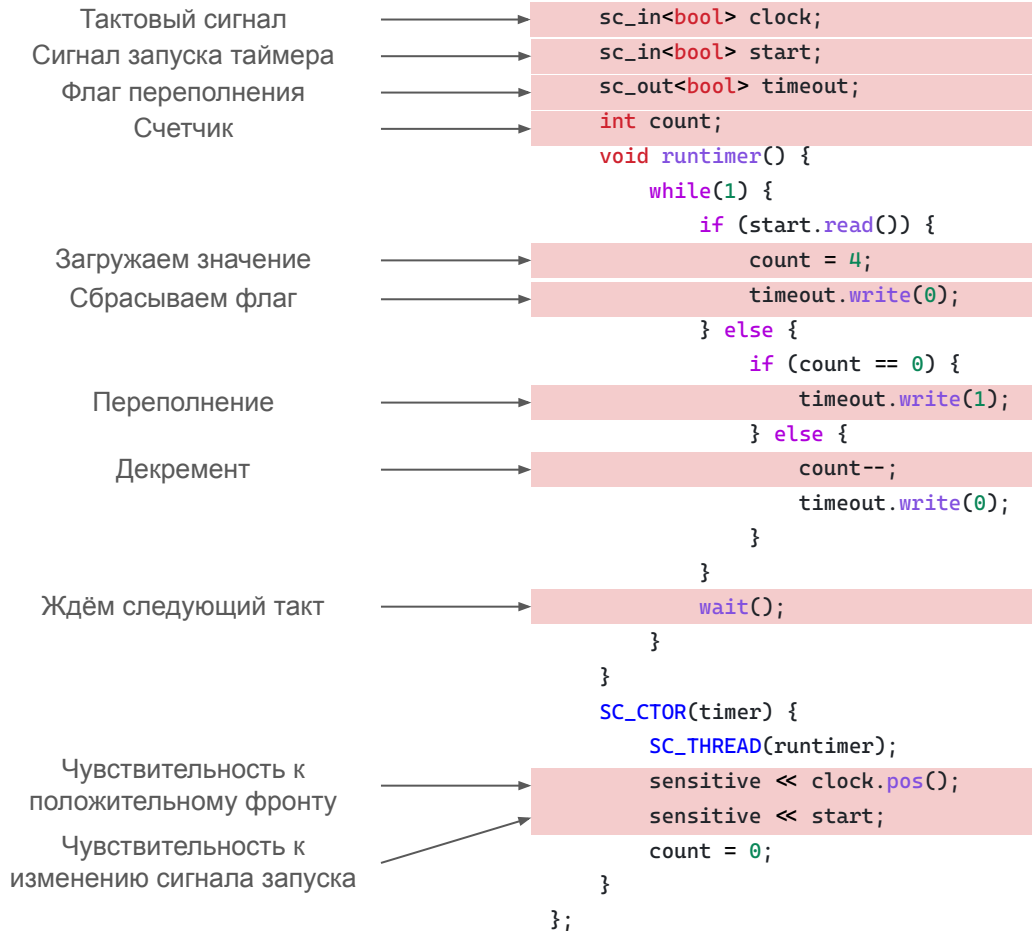
SystemC.clock
SystemC.counter_out

1
0



SystemC. Таймер

- Можем реализовать и по-другому:
 - Засыпает на нужное количество тактов с помощью wait()
 - Просыпается – взводит флаг прерывания



Модель таймера на SystemC

SystemC. Регистр

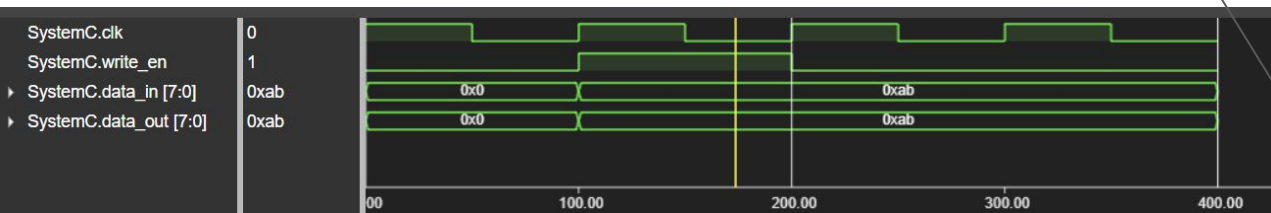
- Реализуем модуль 8-битного регистра с синхронной параллельной записью данных

Разрешение записи
Порт для входного значения
Порт выхода регистра

Синхронная запись по положительному фронту,
комбинаторное чтение

Чувствительность к положительному фронту

Начало передачи данных через 10 ns



```
SC_MODULE(Register) {  
    sc_in<bool> clk;  
    sc_in<bool> write_en;  
    sc_in<uint8_t> addr;  
    sc_in<uint8_t> data_in;  
    sc_out<uint8_t> data_out;  
    uint8_t data;  
  
    void access() {  
        if (write_en.read())  
            data = data_in.read();  
        data_out.write(data);  
    }  
  
    SC_CTOR(Register) : data(0) {  
        SC_METHOD(access);  
        sensitive << clk.pos();  
    }  
};  
  
int sc_main(int argc, char* argv[]) {  
    // ...  
    write_en.write(false);  
    sc_start(10, SC_NS);  
  
    write_en.write(true);  
    data_in.write(0xAB);  
    sc_start(10, SC_NS);  
    // ...  
}
```

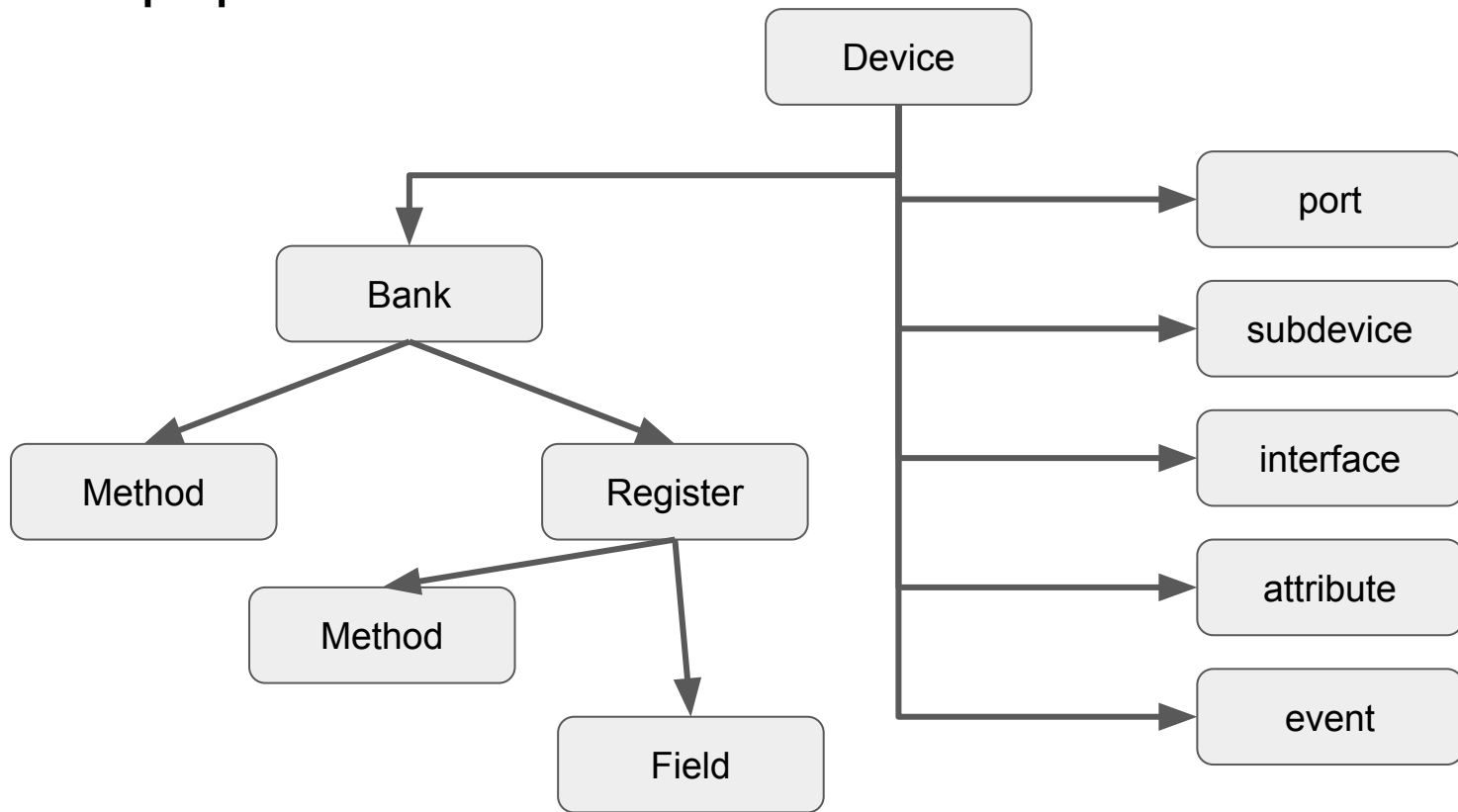
Модель регистра на SystemC

DML



- DSL из Simics
- OpenSource с 2021
- Спроектирован для ускорения прототипирования устройств
- Описание неисполняющих устройств
- Абстракции предметной области на уровне языка
- Компилятор source-to-source
- Позволяет встраивать код на C

Иерархия типов DML



DML example

```
1 dml 1.4;
2
3 device contraption;
4
5 connect wakeup {
6     interface signal;
7 }
8
9 bank config_registers {
10     register cfg1 size 4 @ 0x0000 {
11         field status @ [7:6] is (read, write) {
12             method read() → (uint64) {
13                 local uint2 ret;
14                 ret[0] = enable.val;
15                 ret[1] = coefficient[1] & 1;
16                 return ret;
17             }
18         }
19         field enable @ [8] is (read_unimpl, write) {
20             method write(uint64 val) {
21                 if (this.val == 0 and val == 1) {
22                     wakeup.signal.signal_raise();
23                 } else if (this.val == 1 and val == 0) {
24                     wakeup.signal.signal_lower();
25                 }
26             }
27         }
28     }
29 }
30 }
31
```

Итого

- Что можно сказать про представленные решения?
- SystemC
- DML

Итого

Критерий	QEMU	gem5	SIMICS (DML)
Внутренний DSL	Нет	~Да	Да
Ст-сть разработки	Высокая	Средняя	Низкая
Ст-сть освоения	Высокая	Средняя	Низкая
Отладка	Средняя	Средняя	~Легкая
Переиспользуемость	Низкая	Средняя	Высокая
Открытость/ экосистема	Высокая	Высокая	Низкая

Языки описания аппаратуры

- Hardware Description Languages (HDL)
 - Фокус на описании логической функции
 - САПР используется для *синтеза* оптимизированной логической схемы
- HDL широко применяется в индустрии для прототипирования устройств
- Два основных языка
 - SystemVerilog
 - Разработан Филом Мурби и Прахбу Гоелом в 1984 (Verilog)
 - Стандартизован в 1995 (IEEE)
 - Последний стандарт 2023
 - VHDL 2008
 - Разработан для мин. обороны США в 1983
 - Стандартизован в 1987 (IEEE)
 - Последний стандарт 2019



Моделирование и Синтез

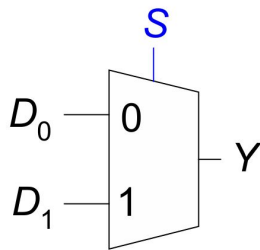
- Моделирование
 - На входы подается воздействие
 - Выходы проверяются на корректность
 - Позволяет сохранить \$\$\$
- Синтез
 - HDL -> netlist (список логических элементов и соединений)
 - Выполняется оптимизация числа элементов

Типы модулей

- Модуль - блок цифровой аппаратуры

```
module mux2x1 (  
    input a, b, sel,  
    output reg y  
);  
    always @(*) begin  
        if (sel)  
            y = b;  
        else  
            y = a;  
        end  
    endmodule
```

Поведенческий



```
module mux2x1_struct (  
    input a, b, sel,  
    output y  
);  
    wire not_sel, and_a, and_b;  
  
    not (not_sel, sel);  
    and (and_a, a, not_sel);  
    and (and_b, b, sel);  
    or (y, and_a, and_b);  
endmodule
```

Структурный

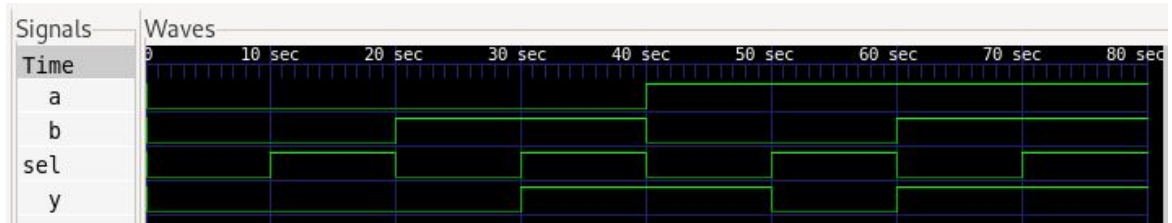
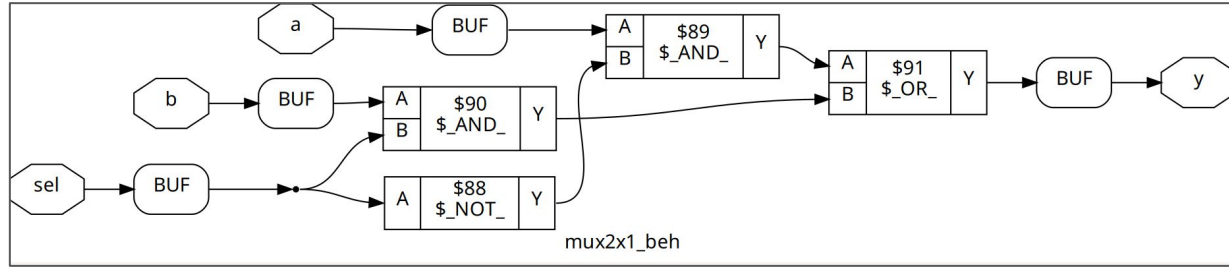
Что дальше?

- Отписанный мультиплексор не очень полезен
- Хочется уметь тестировать и синтезировать модели
- Icarus Verilog
 - Компилятор HDL
 - Симулятор (vvp)
- Yosys
 - Open-source синтезатор
 - Преобразует verilog в netlist
 - Оптимизирует схему
- GtkWave
 - Просто дебажим vcd файлы...



Смоделируем?

```
module mux2x1 (  
    input a, b, sel,  
    output reg y  
);  
    always @(*) begin  
        if (sel)  
            y = b;  
        else  
            y = a;  
        end  
    endmodule
```



Verilator

- Фреймворк для симуляции HDL кода
- “Компилирует” HDL в модель на C++
- Быстрее Icarus Verilog (100x)
- МТ симуляция
- Статический анализ
- Упрощает тестирование моделей
- Поддерживает трассировку (vcd)
- Простая интеграция в CMake



```
10  add_executable(sim main.cc)
11  target_compile_features(sim PRIVATE cxx_std_20)
12  target_compile_options(sim PRIVATE -Wall -Wextra -Wpedantic)
13
14  verilator(
15      sim
16      SOURCES
17      top.v
18      TRACE
19      VERILATOR_ARGS
20      -cc
21      -Wall
22      --exe)
```

Verilator workflow

- Написали просутю модель на Verilog
- Попробуем её исполнить в C++

```
#include "Vmux2x1_beh.h" // generated
int main() {
    auto mux = std::make_unique<Vmux2x1_beh>();
    mux->a = 1;
    mux->b = 0;
    mux->sel = 0;
    mux->eval();
    std::println("y = {}", mux->y);
}
```

```
verilator --cc --exe --build mux2x1_beh.sv mux.cpp
```

```
~/temp/sv
> ./obj_dir/Vmux2x1_beh
y = 1
```

Verilog workflow

- А можно что то сложнее?
- Конечно, можно реализовать RISC-V-32I pipelined CPU

```
int clock = 0;
for (vuint64_t vtime = 1; !Verilated::gotFinish() && vtime ≤ end_time;
    ++vtime)
{
    if (vtime % 8 == 0)
    {
        clock ^= 1;
        top->clk = clock;
        top->eval();
        tracer.dump(vtime);
    }
}
```

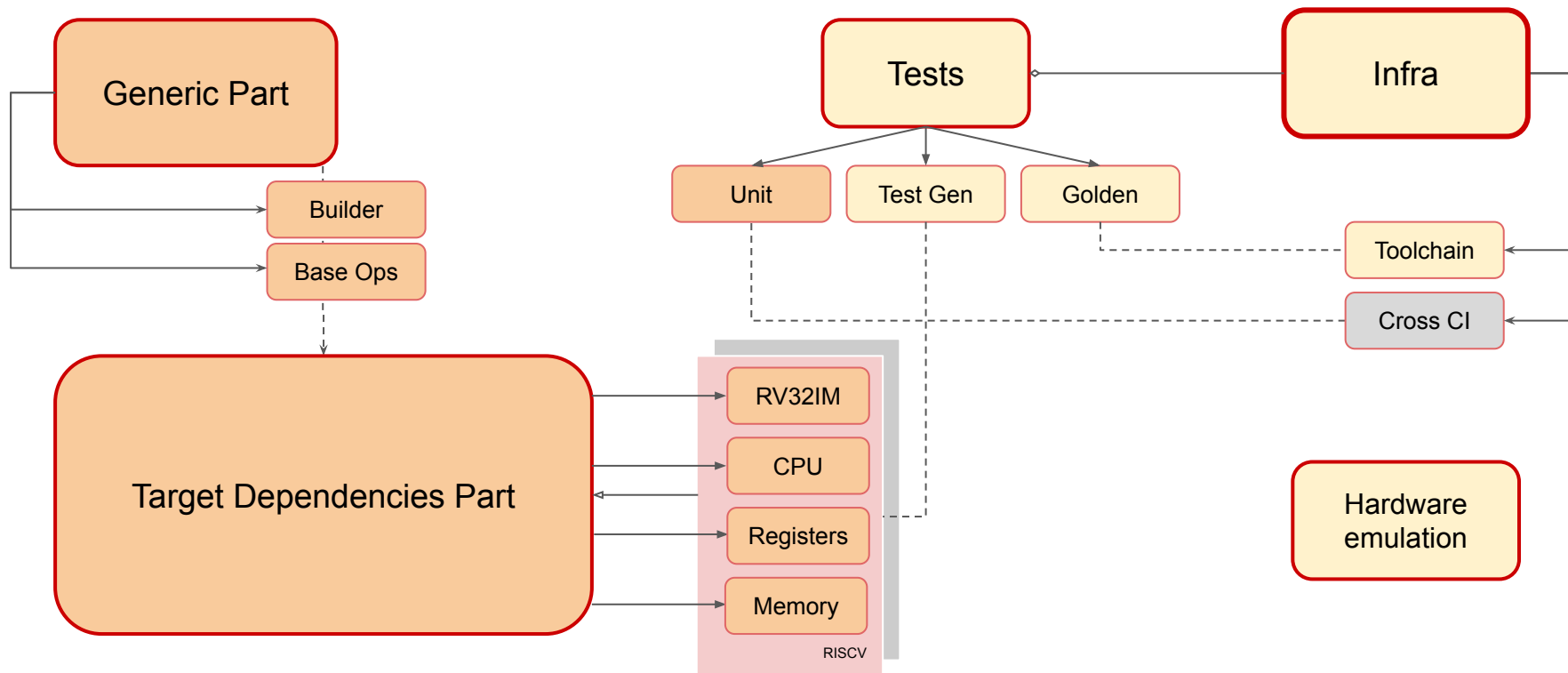
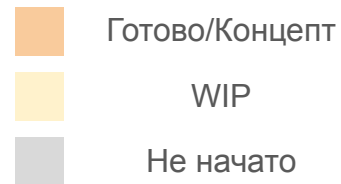
- V adder.v
- V alu.v
- V aludec.v
- V aluop.v
- CMakeLists.txt
- V consts.v
- V controller.v
- V datapath.v
- V dmem.v
- V hazard.v
- V imem.v
- V immSel.v
- main.cc
- V maindec.v
- V mux2.v
- V mux3.v
- V opcodes.v
- V pcReg.v
- V regfile.v
- V riscv.v
- V rPipe.v
- V setForward.v
- V top.v

Verilator?

- Verilator достаточно быстрый
- Однако для больших моделей можно упереться в скорость
 - Моделируем достаточно детально
 - На простой модели RISC-V pipeline - 0.1 MIPS
- Verilog позволяет детально описать поведение (и структуру) модуля

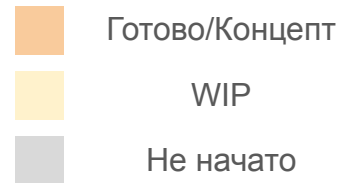
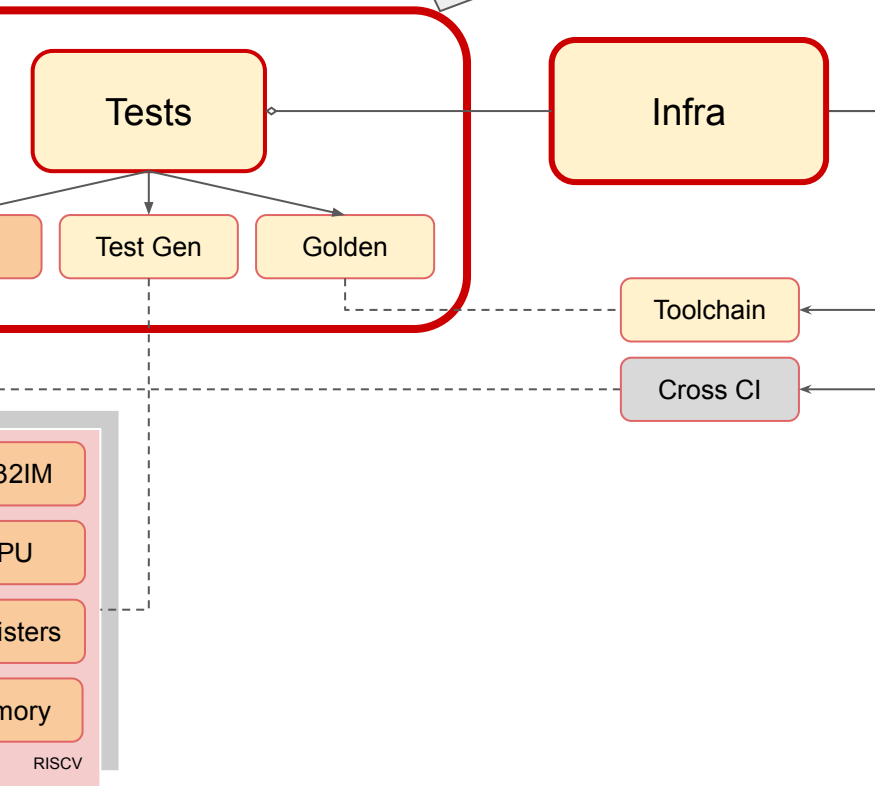
PLoD

Дизайн



Дизайн

Д3-1



Основные концепции:

- Регистр
- Банк регистров
- Устройство
- Композиция устройств
- Шина

Hardware emulation

16550 UART

- Интегральная микросхема, использующаяся для интерфейса последовательной связи
- Используется для преобразования данных от ЦПУ в последовательный поток битов и обратно
- Встроенный 16-байтный FIFO-буфер для TXD/RXD

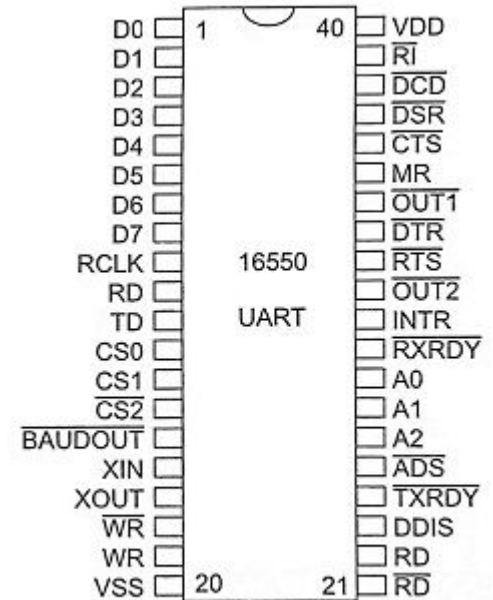


Схема выводов UART 16550

16550 UART

- Интерфейсы можно поделить на следующие группы:
 - Системные
 - Микропроцессорные интерфейсы
 - Прерывания
 - Управление прямым доступом к памяти, DMA
 - Модемные
 - Последовательные
 - Общего назначения

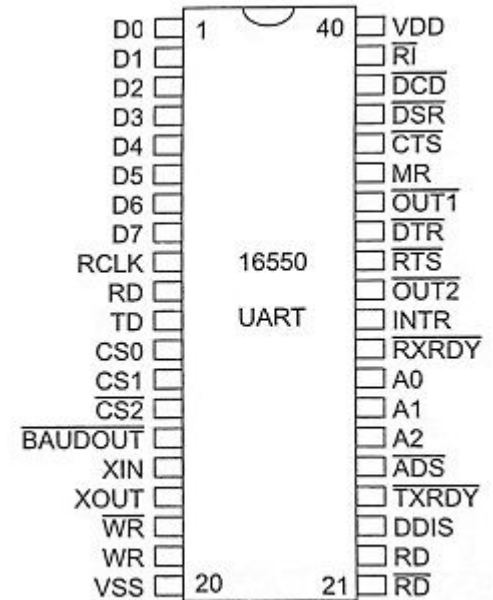


Схема выводов UART 16550

16550 UART

- PIN (интерфейсы):
 - 8-битная шина данных (D0-D7) и управляющие сигналы (CS, RD, WR, A0-A2) для интеграции с шиной процессора
 - Аппаратное управление потоком (RTS/CTS и DTR/DSR)
 - Тактирование (RCLK, BAUDOUT, XIN/XOUT) и прерывания (INTR)
 - Мастер-сброс (MR)
 - Входные адресные шины (A0, A1, A2)
 - Сигнал чтения (RD), сигнал записи (WR)
 - ...

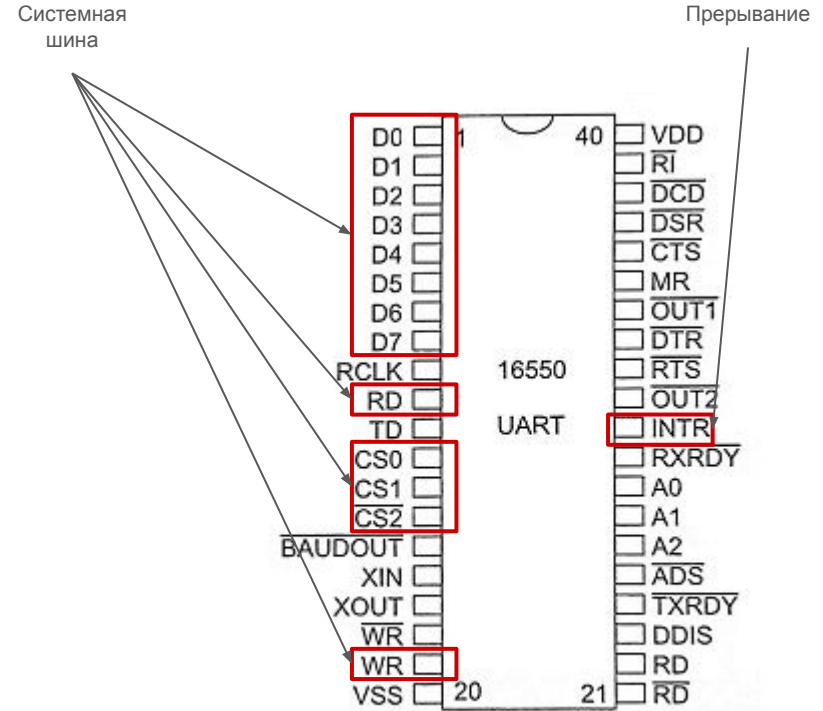


Схема выводов UART 16550

16550 UART

- PIN (интерфейсы):
 - 8-битная шина данных (D0-D7) и управляющие сигналы (CS, RD, WR, A0-A2) для интеграции с шиной процессора
 - Аппаратное управление потоком (RTS/CTS и DTR/DSR)
 - Тактирование (RCLK, BAUDOUT) и прерывания (INTR)
 - Мастер-сброс (MR)
 - Входные адресные шины (A0, A1, A2)
 - Сигнал чтения (RD), сигнал записи (WR)
 - ...

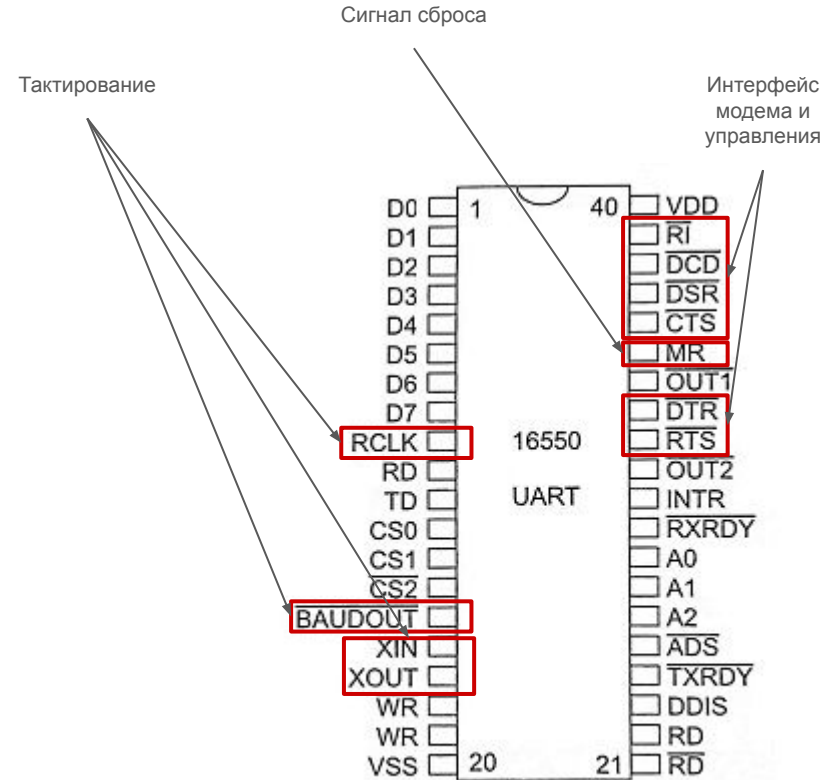
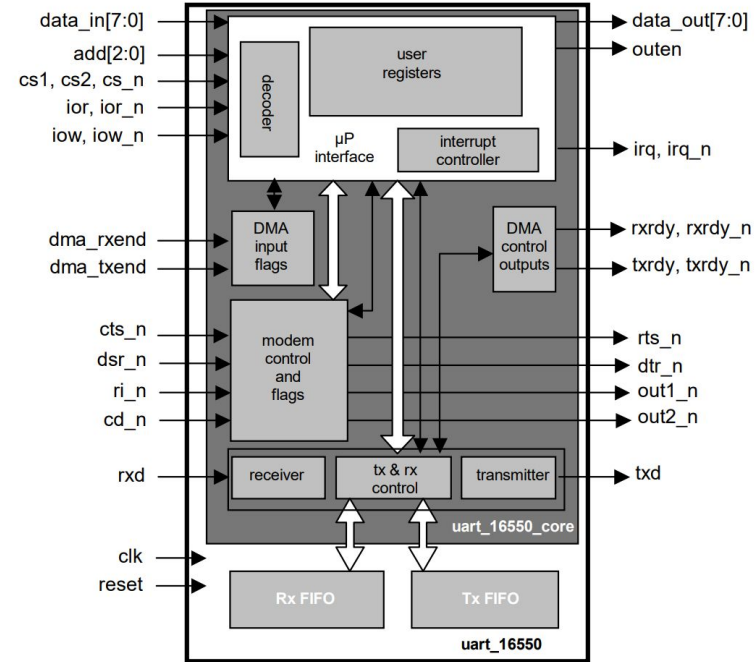


Схема выводов UART 16550

16550 UART

- Регистры нужны для:
 - Программной настройки
 - Контроля состояния
 - Обмена данными
 - Управления прерываниями
- Регистры:
 - Transmit Holding Register (THR)
 - Receive Holding Register (RHR)
 - Interrupt Enable Register (IER)
 - Interrupt Status Register (ISR)
 - FIFO Control Register (FCR)
 - Line Control Register (LCR)
 - Modem Status Register (MSR)
 - Scratch Pad Register (SPR)
 - ...



UART 16550

16550 UART

- [Подробнее](#)

Address & Access type		Register	Bit number									reset value
			7	6	5	4	3	2	1	0		
General Register Set												
000	R	Receiver Holding Register	RHR	Character Received								00
000	W	Transmitter Holding Register	THR	Character to be Transmitted								00
001	R/W	Interrupt Enable Register	IER	DMA Tx End	DMA Rx End	0	0	Modem Status	Receiver Line Status	THR Empty	Data Ready	00
010	R	Interrupt Status Register	ISR	FIFOs enabled	FIFOs enabled	DMA Tx End	DMA Rx End	Interrupt Identification Code			Interrupt Status	01
010	W	FIFO Control Register	FCR	Receiver's FIFO Trigger Level		0	Enable DMA End	DMA mode	Tx FIFO Reset	Rx FIFO Reset	FIFO enable	00
011	R/W	Line Control Register	LCR	DLAB	Set Break	Force Parity	Even Parity	Parity Enable	Stop Bits	Word Length		00 ₁
100	R/W	Modem Control Register	MCR	0	0	0	Loop back	Out 2 / Int. Enable ²	Out 1	RTS	DTR	00
101	R	Line Status Register	LSR	FIFO data Error	Transmitter Empty	THR Empty	Break Interrupt	Framing Error	Parity Error	Overrun Error	Data Ready	60
110	R	Modem Status Register	MSR	CD	RI	DSR	CTS	delta CD	trailing edge RI	delta DSR	delta CTS	00 ₃
111	R/W	Scratch Pad Register	SPR	User Data								00
Registers accesible only when DLAB = 1												
000	R/W	Divisor Latch, Least signif. byte	DLL	Baudrate Divisor's Constant Least Significant Byte								01 ₄
001	R/W	Divisor Latch, Most signif. byte	DLM	Baudrate Divisor's Constant Most Significant Byte								01 ₄
101	W	Prescaler Division	PSD	0	0	0	0	Prescaler's Division Factor				00

16550 UART

- Line Control Register (LCR) отвечает за настройку формата последовательных данных:
 - Длина слова
 - Стоп-биты
 - Чётность
- DLAB (7 бит LCR) – переключатель доступа к регистрам, которые физически расположены по тем же адресам

Address & Access type		Register		Bit number							reset value
				7	6	5	4	3	2	1	
011	R/W	Line Control Register	LCR	DLAB	Set Break	Force Parity	Even Parity	Parity Enable	Stop Bits	Word Length	00 1

Регистр LCR

Реализация UART

- Рассмотрим реализацию на PLoD

```
Register(:thr) {  
    size 0x1  
    offset 0x0  
    type :wo  
    enableIf { lcr.dlab == 0 }  
}
```

```
Register(:ier) {  
    size 0x1  
    offset 0x1  
    enableIf { lcr.dlab == 0 }  
}
```

Реализация регистров 16550 на PLoD

Address & Access type	Register	Bit number								reset value		
		7	6	5	4	3	2	1	0			
General Register Set												
000	W	Transmitter Holding Register	THR	Character to be Transmitted							00	
001	R/W	Interrupt Enable Register	IER	DMA Tx End	DMA Rx End	0	0	Modem Status	Receiver Line Status	THR Empty	Data Ready	00

ДЗ-3 (основное)

- Реализовать таймер (PIT) и контроллер прерываний (PIC) прерываний с помощью языка описания аппаратуры/устройств (любой на выбор):
 - systemc
 - SystemVerilog + Verilator
 - DML
- Интегрировать таймер в ваш функциональный симулятор или автоматически сгенерированную модель из [Protea](#)
- 1 инструкция = 1 такт
- 1 уровень:
 - Симулятор останавливает исполнение при получении прерывания
- 2 уровень:
 - Симулятор корректно обрабатывает прерывание + выводит сообщение (исполнение продолжается)
- 3 уровень (sysregs):
 - Программа сама настраивает таймер при запуске, (период + обработчик)
 - Обработчик - вывод на экран через syscall write
- Доп задача *:
 - Реализовать простую DES

Литература

1. [SystemC User's Guide](#)
2. [intel/device-modeling-language · GitHub](#)
3. [Verilator open-source SystemVerilog simulator and lint system · GitHub](#)
4. [UART 16550 IP Datasheet](#)
5. [16550 UART Pin Diagram](#)
6. [riscv/riscv-isa-manual: RISC-V Instruction Set Manual · GitHub](#)
7. [Ratified Specifications - RISC-V International](#)